

Mid-Term Group Portfolio/Project – 1 -2026

Job Shop Scheduling Problem Using Genetic Algorithms

Objective: Develop and implement a Genetic Algorithm (GA) in Python to solve a combinatorial optimization problem in the manufacturing sector. Specifically, you will address the Job Shop Scheduling Problem (JSSP), a classic scheduling problem of assigning and sequencing job operations on machines to minimise the overall completion time.

- **Group Policy:** Pre-approved groups only. **Submissions outside the group list will NOT be accepted.**
- **Deadline:** September 17th (Thursday) at 12:00 (midday).
- **Submission Channel:** Canvas.
- **Academic Integrity:** Plagiarism is handled per OsloMet policy. You can find more about it at <https://student.oslomet.no/en/cheating>
- **Use of Artificial Intelligence (AI):** You may use AI tools for coding as permitted by university guidelines, but **NO AI tools are permitted to write the report.** You can find the OsloMet policy at: <https://student.oslomet.no/en/veiledning-kunstig-intelligens-studentoppgaver>
- **Submission Procedure:** Every group must **submit only ONE submission** by any group member. We will download the code immediately after the submission deadline to prevent later updates.

The Job Shop Scheduling Problem (JSSP)

The Job Shop Scheduling Problem (JSSP) involves scheduling a set of jobs (n) on a set of machines (m). Each job consists of an ordered sequence of operations, and each operation must be processed on a specified machine for a given processing time. Each machine can process only one operation at a time, and the required order of operations within each job must be respected. The output is a feasible schedule specifying when each operation starts and finishes. The usual objective is to minimize the makespan, i.e., the completion time of the last operation.

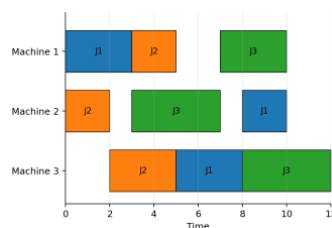


Fig. 1: A JSSP Schedule

The objective is to minimize the total completion time of all jobs, known as the **Makespan** ($C_{\max}$):

$$\min C_{\max} = \min \left(\max_{i \in \{1, \dots, n\}} C_i \right)$$

where C_i represents the completion time of the final operation of job i .

Task Details

- **Problem Definition:**
 - **Dataset Source:** Use the public JSPLib Job Shop Scheduling benchmark library: <https://scheduleopt.github.io/benchmarks/jsplib/>

The selected instance files provide the number of jobs and machines and, for each job, the ordered machine-processing-time pairs for its operations. JSPLib also provides best-known bounds/solutions that you can use to assess solution quality.

- **Problem Categories:** Define three groups of scenarios, each with 2 instances. Use benchmark instances from the Lawrence (*la*) family within [Lawrence1984](#) GitHub folder in the above JSPLib link. The groups should be categorized as:
 1. **Small:** select 2 instances from *la01-la05*, each with 10 jobs and 5 machines.
 2. **Medium:** select 2 instances from *la16-la20*, each with 10 jobs and 10 machines.
 3. **Large:** select 2 instances from *la31-la35*, each with 30 jobs and 10 machines.

- **Chromosome Decoding and Schedule Building Algorithm (SBA)**

A raw GA chromosome (such as an operation-based sequence array) does not explicitly form a schedule. Your Python program must convert the chromosome into a valid, non-overlapping schedule using a **Schedule Building Algorithm (SBA)** before evaluating the fitness function:

- **Decoding:** Read the chromosome sequence.
 - **Resource Allocation:** Route operations to target machines according to precedence constraints.
 - **Conflict Resolution:** Select a building technique—such as a **Semi-Active**, **Active**, or **Giffler-Thompson** schedule builder—to place operations into the earliest valid idle time slots on the designated machine.
 - **Makespan Calculation:** Extract $C_{\max}$ from the generated Gantt schedule.
- **Mandatory Statistical Performance Metrics:**

Because Evolutionary Algorithms are stochastic, **single-run results are unacceptable**. You **must run every experiment for a minimum of 10–30 independent runs** and calculate:

 - **Best ($C_{\max}^{\min}$):** Peak solution quality found across all runs.
 - **Worst ($C_{\max}^{\max}$):** Worst-case performance bound.
 - **Average (μ):** Measure of central tendency across runs.
 - **Standard Deviation (σ):** Indicator of algorithm stability and robustness.
 - **Execution Time (seconds):** Computational efficiency.
 - **Convergence Rate:** Generation number at which the best value stabilizes.
 - **Comparison and Analysis:**
 - Using the above metrics, compare and analyze your implementation using the three categories of test problems.
 - You also need to test and analyse all the test problems **for three sets of GA parameters** (population size, generation number, crossover probability, and mutation probability).

Deliverables

- a. **Code Implementation:**
 - i. Provide a GitHub repository containing clean Python code (using libraries such as `matplotlib` for Gantt rendering, `numpy`, or `pandas`), executable scripts, step-by-step installation instructions in `README.md`, and inline documentation explaining your

chromosome representation and SBA logic. The README must include instructions for installing any dependencies and running the algorithms on the provided example data.

- ii. **Include the GitHub link in the submitted report and ensure it is shareable and executable.**
- iii. The repository should be well-organized, and the code should be commented on or accompanied by notes explaining key parts of the GA implementation, including the chromosome representation, genetic operators, and the schedule-building/decoding function that converts a chromosome into a feasible JSSP schedule.
- iv. **The code must be executable and generate the outputs.**

b. Report

- i. You should deliver your report in a PDF format through Canvas.
- ii. The project report should be between 1000-1500 words.
- iii. **The report must mention the Group No as published in the Canvas.**
- iv. The report should at least discuss the points below:
 - 1. A detailed description of the complete GA design. Describe your implementation in sufficient detail for others to replicate your work. This includes the individual representation, initialization strategy, selection strategy, crossover and mutation operators, termination condition, and the SBA algorithm used to transform each chromosome into a feasible JSSP schedule.
 - 2. Include pseudocode or a clear step-by-step explanation of the decoder, and explain how precedence and machine-capacity constraints are enforced.
 - 3. Define and explain the objective/fitness function. Explain how the makespan is derived from the decoded schedule and how this value is used by the GA for fitness evaluation and selection.
 - 4. Show one example of chromosome decoding, from genotype to schedule. For a small benchmark instance or a reduced illustrative example, show the chromosome, the operation order generated by the decoder, the resulting start/finish times (or a small schedule table), and the corresponding Gantt chart.
 - 5. Using the specified metrics, compare and analyze your implementation across our three categories of test problems.
 - 6. A table summarizing the required time to find the solution for all the test problems using your three defined parameter sets.
 - 7. What are your findings, analysis, and conclusions regarding the correlation between the parameters, based on the results achieved?
 - 8. Describe the effects of these parameter values in the early and later stages of the evolutionary cycle.